

Homework 02 — STAT 432 (Fall 2026)

Name: Owen Park

NetID: owenp3

Questions 1–5 are done in Python. Each part reports the numbers *and* what they mean.

```
In [1]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import numpy.linalg as la

# Seeds are per question (43202 for Q1, 43203 for Q2); rng is re-created where
```

Question 1: Training and test error under a fixed design

$n=100$, $p=20$, $\beta_j = 0.4^{\sqrt{j}}$, X_{all} fixed, 200 runs, seed 43202 .

1a

Average train/test MSE over 200 runs for $m=0..20$, plot both vs m , check train MSE never increases within a run.

```
In [2]: # Q1a
n, p, R = 100, 20, 200
rng = np.random.default_rng(43202)
X_all = rng.normal(size = (n, p))
j = np.arange(1, p + 1)
beta_j = 0.4 ** np.sqrt(j)
mu = X_all @ beta_j

e = rng.normal(size = (R, n))
e_star = rng.normal(size = (R, n))
Y = mu + e
Y_star = mu + e_star

M = np.arange(0, p+1)

mse_train = np.zeros((R, p + 1))
mse_test = np.zeros((R, p + 1))
H = []
for m in M:
    X_m = np.column_stack([np.ones(n), X_all[:, :m]])
    H_m = X_m @ np.linalg.solve(X_m.T @ X_m, X_m.T)
    H.append(H_m)
    Yhat = Y @ H_m
    mse_train[:, m] = np.mean((Y - Yhat) ** 2, axis=1)
    mse_test[:, m] = np.mean((Y_star - Yhat) ** 2, axis=1)
```

```

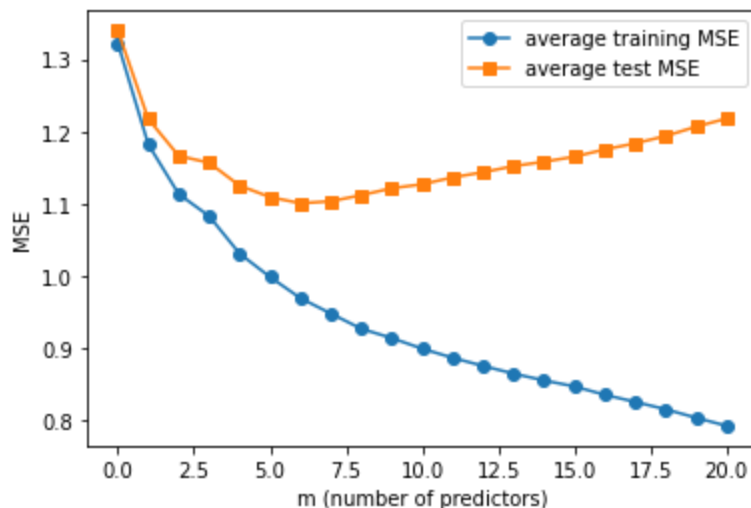
avg_train = mse_train.mean(axis=0)
avg_test = mse_test.mean(axis=0)

# numerical check: training MSE never increases within a run
never_increases = np.all(np.diff(mse_train, axis=1) <= 1e-10)
print("train MSE non-increasing in every run:", never_increases)
print("m minimizing average test MSE:", M[avg_test.argmax()])

plt.plot(M, avg_train, "o-", label="average training MSE")
plt.plot(M, avg_test, "s-", label="average test MSE")
plt.xlabel("m (number of predictors)"); plt.ylabel("MSE"); plt.legend(); plt.s

```

train MSE non-increasing in every run: True
m minimizing average test MSE: 6



1b

$B_m^2 = \|(I - H_m) \mu\|^2$. Add $E[\text{train}] = B_m^2/n + 1 - (m+1)/n$ and $E[\text{test}] = B_m^2/n + 1 + (m+1)/n$ to the plot.

```

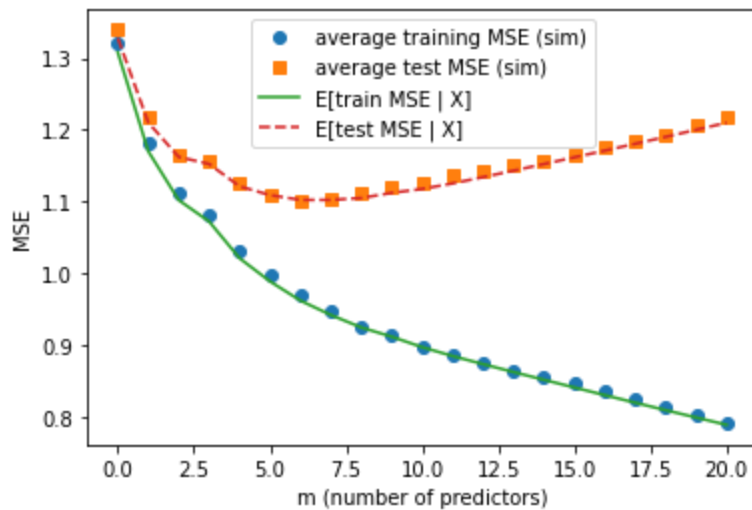
In [3]: # Q1b
B_m2 = np.array([la.norm(mu - H[m] @ mu) ** 2 for m in M])
exp_train = B_m2 / n + 1 - (M + 1) / n
exp_test = B_m2 / n + 1 + (M + 1) / n

print("max |sim avg - theory| train:", np.abs(avg_train - exp_train).max().round(4))
      " test:", np.abs(avg_test - exp_test).max().round(4))
print("m minimizing theoretical test MSE:", M[exp_test.argmax()])

plt.plot(M, avg_train, "o", label="average training MSE (sim)")
plt.plot(M, avg_test, "s", label="average test MSE (sim)")
plt.plot(M, exp_train, "-", label="E[train MSE | X]")
plt.plot(M, exp_test, "--", label="E[test MSE | X]")
plt.xlabel("m (number of predictors)"); plt.ylabel("MSE"); plt.legend(); plt.s

max |sim avg - theory| train: 0.0143 test: 0.0119
m minimizing theoretical test MSE: 7

```

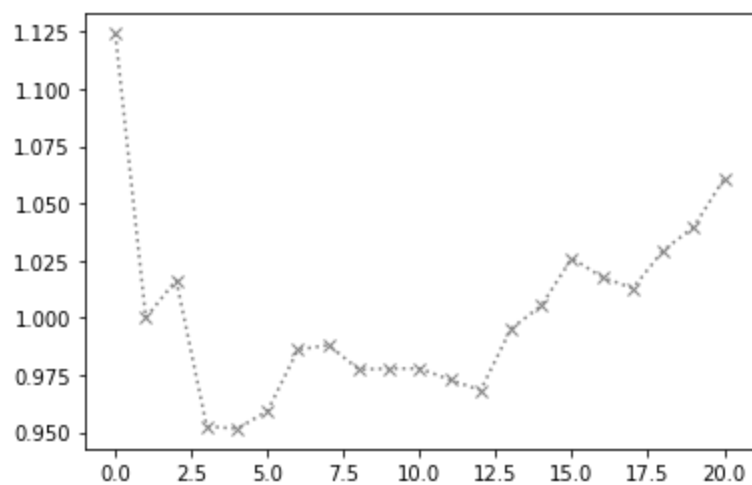


1c

Overlay one run's test MSE curve. Why is it rougher than the average? Why can't train MSE alone pick m ?

```
In [4]: # Q1c
one_run = mse_test[0] # run r = 0
plt.plot(M, one_run, "x:", color="gray", label="test MSE, single run (r = 0)")
print("single run: argmin m =", M[one_run.argmin()],
      "| upticks between consecutive m:", int((np.diff(one_run) > 0).sum()))
print("per-run argmin m, counts for m = 0..20:", np.bincount(mse_test.argmax(axis=0)))
print("train MSE argmin is m = 20 in every run:", np.all(mse_train.argmax(axis=0) == 20))
```

```
single run: argmin m = 4 | upticks between consecutive m: 12
per-run argmin m, counts for m = 0..20: [ 0  5 14  9 22 18 30 29 14 11  7 10
  5  3  4  2  3  5  4  2  3]
train MSE argmin is m = 20 in every run: True
```



Compared to the one that have all average of 200 runs, a single run test's MSE only have one case, and it depends on the random variable a lot. Also the estimation-variance has a term $(m+1)/n$ is negative on training MSE, which can't explain the test MSE minimum is near $m = 6, 7$.

Question 2: Prediction error at one target point

$n=100$, $p=6$, cosine design (orthogonal columns), $\beta = 0.5$ each, $x_0 = (0,1,0,0,1,0)$, seed 43203.

2a

μ_0 , theoretical squared bias, expected squared error for $m=0..6$ via the given formula.
Where does it change / stay flat?

```
In [5]: # Q2a
n, p = 100, 6
i = np.arange(1, n + 1)[: , None]
j = np.arange(1, p + 1)[None, :]
X_all = np.sqrt(2) * np.cos(np.pi * j * (i - 0.5) / n)
beta = np.full(p, 0.5)
x0 = np.array([0, 1, 0, 0, 1, 0.0])
sigma2 = 1.0
ms = np.arange(0, p + 1)

mu0 = x0 @ beta
bias2 = np.array([(x0[m] @ beta[m]) ** 2 for m in ms])
var = np.array([sigma2 / n * (1 + np.sum(x0[:m] ** 2)) for m in ms])
theory = bias2 + var

print("mu0 =", mu0)
print(pd.DataFrame({"m": ms, "bias2": bias2, "var": var, "expected_sq_err": theory}))
print("smallest m minimizing theoretical error:", ms[theory.argmin()])

mu0 = 1.0
      m  bias2   var  expected_sq_err
0  0    1.00  0.01             1.01
1  1    1.00  0.01             1.01
2  2    0.25  0.02             0.27
3  3    0.25  0.02             0.27
4  4    0.25  0.02             0.27
5  5    0.00  0.03             0.03
6  6    0.00  0.03             0.03
smallest m minimizing theoretical error: 5
```

The expected square error only changes when $m = 2,5$, which is the part that bias2 changes, which is also the only part that x_0 has 1 on it. Bias2 part shows where it eliminated non zero values from X_0 . Also, multiplying from x_0 which is 0 wouldn't change anything, which also you can see on difference on variance.

2b

200 runs: $(\mu_{\text{hat}} - \mu_0)^2$, plot sim averages vs theory, report the smallest m minimizing the theoretical error.

```
In [6]: # Q2b
rng = np.random.default_rng(43203)
```

```

R = 200

eps = rng.normal(size=(R, n))
Y = X_all @ beta + eps

sq_err = np.zeros((R, p + 1))
for m in ms:
    X_m = np.column_stack([np.ones(n), X_all[:, :m]])
    x0_m = np.concatenate([[1.0], x0[:m]])
    w = np.linalg.solve(X_m.T @ X_m, x0_m)
    muhat0 = Y @ (X_m @ w)
    sq_err[:, m] = (muhat0 - mu0) ** 2

avg_sq = sq_err.mean(axis=0)
print("simulation average:", avg_sq.round(4))
print("max |sim - theory|:", np.abs(avg_sq - theory).max().round(4))
print("smallest m minimizing theoretical error:", ms[theory.argmin()])

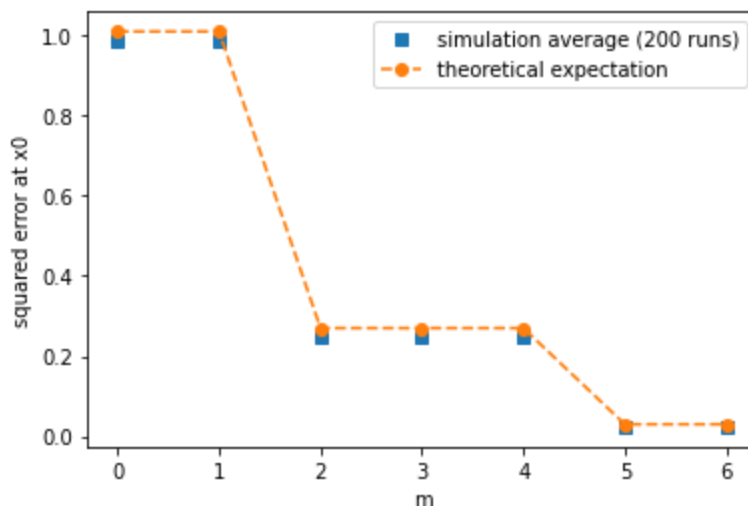
plt.plot(ms, avg_sq, "s", label="simulation average (200 runs)")
plt.plot(ms, theory, "o--", label="theoretical expectation")
plt.xlabel("m"); plt.ylabel("squared error at x0"); plt.legend(); plt.show()

```

```

simulation average: [0.9839 0.9839 0.2464 0.2464 0.2464 0.0237 0.0237]
max |sim - theory|: 0.0261
smallest m minimizing theoretical error: 5

```



2c

Why do only predictors 2 and 5 matter at x_0 ? Contrast with the row-averaged test MSE in Q1.

Even though all six regression coefficients are nonzero, if you multiply zero, it doesn't contribute to mean response; thus, only 2, 5 which is 1 only contributes to it. In question 1, they averaged so not every predictors are not zero, so they all contributes.

Question 3: The optimism correction

Theory question — derivations in markdown, a small code cell for the arithmetic.

3a

Derive $E[\text{MSE}_{\text{train}}|\mathbf{X}] = B^2/n + \sigma^2(1 - (p+1)/n)$ and $E[\text{MSE}_{\text{test}}|\mathbf{X}] = B^2/n + \sigma^2(1 + (p+1)/n)$.

Write $\mathbf{y} = \boldsymbol{\mu} + \boldsymbol{\epsilon}$ and $\mathbf{y}^* = \boldsymbol{\mu} + \boldsymbol{\epsilon}^*$. Since $\mathbf{H}$ is symmetric and idempotent, so is $\mathbf{I} - \mathbf{H}$.

Training. The residual is $\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\boldsymbol{\mu} + (\mathbf{I} - \mathbf{H})\boldsymbol{\epsilon}$, so

$$\|\mathbf{y} - \mathbf{H}\mathbf{y}\|^2 = \|(\mathbf{I} - \mathbf{H})\boldsymbol{\mu}\|^2 + 2\boldsymbol{\mu}^\top(\mathbf{I} - \mathbf{H})\boldsymbol{\epsilon} + \boldsymbol{\epsilon}^\top(\mathbf{I} - \mathbf{H})\boldsymbol{\epsilon}.$$

Taking expectations given $\mathbf{X}$: the first term is B^2 ; the $E[2\boldsymbol{\mu}^\top(\mathbf{I} - \mathbf{H})\boldsymbol{\epsilon}] = 0$ because $E(\boldsymbol{\epsilon} | \mathbf{X}) = \mathbf{0}$; the quadratic form has mean $\sigma^2 \text{tr}(\mathbf{I} - \mathbf{H}) = \sigma^2(n - p - 1)$. Dividing by n ,

$$E(\text{MSE}_{\text{train}} | \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 - \frac{p+1}{n}\right).$$

Test. Now $\mathbf{y}^* - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}$, a sum of a fixed vector and two independent mean-zero vectors. Every cross term therefore has expectation zero, and

$$E\|\mathbf{y}^* - \mathbf{H}\mathbf{y}\|^2 = B^2 + E\|\boldsymbol{\epsilon}^*\|^2 + E\|\mathbf{H}\boldsymbol{\epsilon}\|^2 = B^2 + n\sigma^2 + \sigma^2 \text{tr}(\mathbf{H}^\top \mathbf{H}) = B^2 + n\sigma^2 + \sigma^2(p+1),$$

using $\mathbf{H}^\top \mathbf{H} = \mathbf{H}$ and $\text{tr}(\mathbf{H}) = p+1$. Dividing by n ,

$$E(\text{MSE}_{\text{test}} | \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 + \frac{p+1}{n}\right).$$

The only difference is the sign of $\sigma^2(p+1)/n$: on training data the fit absorbs part of the noise it is scored on; on test data the fitted noise is an extra penalty.

3b

Expected optimism; plug in $n=120$, $p=4$, $B^2/n=0.04$, $\sigma^2=1$. Why can a realized pair violate the ordering?

Q3b

Subtracting the two expectations, the bias terms cancel and

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} | \mathbf{X}) = \frac{2\sigma^2(p+1)}{n}.$$

With $n = 120$, $p = 4$, $B^2/n = 0.04$, $\sigma^2 = 1$:

$$E(\text{MSE}_{\text{train}}) = 0.04 + 1 - 5/120 = 0.9983,$$

$$E(\text{MSE}_{\text{test}}) = 0.04 + 1 + 5/120 = 1.0817, \text{ difference} = 10/120 = 0.0833.$$

These are expectations over ϵ and ϵ^* . For one realized pair, MSE_{test} depends on a fresh ϵ^* that can happen to be small, and $\text{MSE}_{\text{train}}$ on an ϵ that can happen to be large, so a single draw can show test below train. The gap of 0.083 is small next to the run-to-run spread of either MSE (each is an average of 120 squared errors with sd of order $\sigma^2 \sqrt{2/n} \approx 0.13$), so the reversal is not rare. Only the average over many pairs must respect the ordering.

3c

RSS=116.4, sigma^2hat=0.96: compute MSEhat_test and C_p, show the relation, why minimizing C_p = minimizing corrected test MSE.

```
In [7]: # Q3c
n, p, RSS, s2 = 120, 4, 116.4, 0.96
mse_hat = (RSS + 2 * (p + 1) * s2) / n
Cp = RSS / s2 - n + 2 * (p + 1)
print("corrected test MSE:", mse_hat, "| Cp:", Cp, "| s2*(Cp + n)/n:", s2 * (Cp + n))
```

corrected test MSE: 1.05 | Cp: 11.2500000000000014 | s2*(Cp + n)/n: 1.05

$$\hat{\text{MSE}}_{\text{test}} = (116.4 + 2 \cdot 5 \cdot 0.96)/120 = 126/120 = 1.05 \text{ and}$$

$$C_p = 116.4/0.96 - 120 + 10 = 11.25.$$

Multiplying C_p by $\hat{\sigma}^2$ gives $\text{RSS} - n\hat{\sigma}^2 + 2(p+1)\hat{\sigma}^2 = n\hat{\text{MSE}}_{\text{test}} - n\hat{\sigma}^2$, so

$$\hat{\text{MSE}}_{\text{test}} = \hat{\sigma}^2 \frac{C_p + n}{n} = \frac{\hat{\sigma}^2}{n} C_p + \hat{\sigma}^2,$$

which checks: $0.96 \cdot 11.25/120 + 0.96 = 1.05$. When n and $\hat{\sigma}^2$ are the same for every candidate model, this is an increasing linear map from C_p to the corrected test MSE, so the model with the smallest C_p is exactly the model with the smallest corrected test MSE. Both are the training RSS plus the optimism correction $2(p+1)\hat{\sigma}^2$ from part (b), just on different scales.

Question 4: Comparing Mallows' C_p, AIC, and BIC

data/diabetes.csv, rows 1–370 only, n=370. Model A (7), Model B (A + s6, 8), full (10).

```
In [8]: dat = pd.read_csv("data/diabetes.csv")
train = dat.iloc[:370]
test = dat.iloc[370:]
print(dat.shape, train.shape, test.shape)
print(list(dat.columns))
```

```
(442, 11) (370, 11) (72, 11)
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6', 'y']
```

4a

Fit the three OLS models. Report p, RSS, residual df of the full model, $\sigma^2_{\text{hat}} = \text{RSS}_{\text{full}} / (n - 11)$.

```
In [9]: # Q4a
n = len(train)
y = train["y"].to_numpy()

def fit_rss(cols):
    X = np.column_stack([np.ones(n), train[cols].to_numpy()])
    b, *_ = np.linalg.lstsq(X, y, rcond=None)
    return float(np.sum((y - X @ b) ** 2))

A = ["bmi", "bp", "s5", "sex", "s1", "s2", "s4"]
B = A + ["s6"]
FULL = ["age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6"]

RSS = {"A": fit_rss(A), "B": fit_rss(B), "full": fit_rss(FULL)}
P = {"A": 7, "B": 8, "full": 10}
df_full = n - (P["full"] + 1)
s2_hat = RSS["full"] / df_full

print(f'n = {n}, residual df of full model = {df_full}, sigma2_hat = {s2_hat:.2f}')
for k in ["A", "B", "full"]:
    print(f'Model {k}: p = {P[k]}, RSS = {RSS[k]:.2f}')

n = 370, residual df of full model = 359, sigma2_hat = 3034.684
Model A: p = 7, RSS = 1098908.57
Model B: p = 8, RSS = 1089717.06
Model full: p = 10, RSS = 1089451.72
```

4b

C_p, reduced AIC, reduced BIC for A and B. Which model does each criterion pick?

```
In [10]: # Q4b
rows = []
for k in ["A", "B"]:
    p, r = P[k], RSS[k]
    rows.append({"model": k, "p": p, "RSS": r,
                 "Cp": r / s2_hat - n + 2 * (p + 1),
                 "AIC": n * np.log(r / n) + 2 * (p + 1),
                 "BIC": n * np.log(r / n) + (p + 1) * np.log(n)})
crit = pd.DataFrame(rows).set_index("model")
print(crit.round(3))
for c in ["Cp", "AIC", "BIC"]:
    print(f'{c} selects Model {crit[c].idxmin()}')
```

	p	RSS	Cp	AIC	BIC
model					
A	7	1098908.566	8.116	2974.640	3005.948
B	8	1089717.057	7.087	2973.532	3008.754

Cp selects Model B
AIC selects Model B
BIC selects Model A

4c

Fit gain from s6 vs the one-parameter penalty under each criterion. Why doesn't selection = true model?

Adding **s6** lowers RSS from 1098908.6 to 1089717.1. On the AIC/BIC scale that improvement is $n \log(\text{RSS}_A / \text{RSS}_B) = 3.11$; on the Cp scale it is $(\text{RSS}_A - \text{RSS}_B) / \sigma^2_{\text{hat}} = 3.03$. Each criterion charges one extra parameter: 2 for Cp, 2 for AIC, $\log(370) = 5.91$ for BIC. A gain of about 3 beats a penalty of 2, so Cp and AIC prefer Model B, but it does not beat 5.91, so BIC keeps Model A. The criteria disagree only because they price the same one-parameter gain differently; the data are the same. Selecting a model says it has the best estimated prediction error (or the best penalized fit) among the candidates we tried. It does not say the true mean is linear in those predictors, that the omitted predictors have zero coefficients, or that a model outside the candidate list is not better. With a different sample the small 3-versus-2 margin could easily flip, so the choice of **s6** is a near tie, not evidence about the data-generating process.

Question 5: Validation and final test data

Rows 1–370 train, 371–442 final test. Eleven nested models in the order age, sex, bmi, bp, s1, ..., s6.

5a

First step that misuses the test data; why the min of eleven test MSEs is too favorable.

Step 3 is the first misuse: choosing the model with the smallest test MSE turns the final test data into selection data. Steps 1 and 2 are fine on their own (fitting used only training rows; computing eleven test MSEs is a legitimate evaluation of eleven fixed models). But the minimum of eleven noisy estimates is biased downward: each test MSE is the true error plus sampling noise on 72 rows, and picking the smallest one picks the model whose noise happened to be most negative. The reported number then estimates the error of the best-looking model on this particular test set, not the future error of the procedure "fit eleven models and pick the best".

5b

Rewrite with 5-fold CV inside the training data: how m is chosen, what is refitted, when the test data may be touched.

Use only rows 1-370. Split them into 5 folds. For each $m = 0..10$ and each fold k : fit the m -predictor model on the other four folds, predict the held-out fold, record the squared errors. Average over the five folds to get $CV(m)$, and choose $m_{\text{hat}} = \text{argmin } CV(m)$ (or the smallest m within one standard error of the minimum). Refit the m_{hat} -predictor model once on all 370 training rows. Only now touch rows 371-442: compute the test MSE of that single refitted model, once, and report it. The test data are used exactly one time, after every choice has been made.

```
In [11]: # Q5b
```

5c

After peeking at all eleven test MSEs: what can still be reported, what new data is needed.

The analyst can still report all eleven test MSE values with the statement that the final test set was used to compare them, so the minimum is an optimistic (selection-biased) estimate rather than an honest one; can report the CV-based selection and CV error from the training data; and can report which m was chosen and how. What cannot be recovered is an unbiased estimate of the selected model's future error, because no data remain that the selection did not see. A new final evaluation needs fresh observations that were not used for fitting or for choosing m , e.g. a newly collected test set or a portion of the data that was locked away before the analysis began.